5.4 Testowanie Autonomiczne: Generowanie kompleksowych scenariuszy E2E i testów regresyjnych przez agentów na podstawie zmian w kodzie

Moduł 5: Agenty AI w procesach CI/CD i DevOps

Krzysztof Rytlewski · 2026

Czym jest testowanie autonomiczne i gdzie leży jego wartość?

Testowanie autonomiczne to podejście, w którym agent AI analizuje zmiany w kodzie i automatycznie generuje lub aktualizuje testy — bez ręcznego pisania scenariuszy przez programistę.

Problem tradycyjnego przepływu: testy są pisane pobieżnie, odkładane na później lub pomijane pod presją czasu. Agent AI rozwiązuje to, generując testy natychmiast po wykryciu zmiany.

Typ testu	Zakres	Narzędzia
Jednostkowe	Pojedyncza funkcja / serwis	Jest, Vitest
Integracyjne	Endpoint API + baza danych	Jest + Supertest + MongoDB in-memory
E2E	Cały przepływ: UI → API → baza	Playwright
Regresyjne	Zestawy E2E uruchamiane po każdej zmianie	Playwright + GitHub Actions

Analiza diff'a i generowanie testów jednostkowych

Punkt wyjścia agenta jest identyczny jak w agencie code review: pobranie diff'a PR przez GitHub API. Różnica leży w prompcie — zamiast szukać błędów, LLM ma wygenerować kod testów.

Agent filtruje zmienione pliki do tylko `.js` / `.ts` (bez plików testowych), następnie dla każdego pliku wysyła diff do LLM z instrukcjami:

- każda eksportowana funkcja musi mieć co najmniej jeden test happy path i jeden test błędu,
- zewnętrzne zależności (MongoDB, OpenAI API) mockujemy przez `jest.mock()`,
- używamy `describe/it/expect` z matcherami Jest,
- zwróć wyłącznie kod pliku testowego.

Wygenerowane pliki testowe agent commituje na gałąź PR, co wyzwala automatyczne uruchomienie testów.

Generowanie testów E2E z Playwright — instalacja

Playwright to nowoczesne narzędzie do testów E2E wspierające Chrome, Firefox i Safari. Testy symulują rzeczywistego użytkownika klikającego po interfejsie Angular.

```
npm init playwright@latest
# Wybierz: TypeScript, folder tests/e2e, GitHub Actions workflow
```

Agent AI generuje scenariusze Playwright bazując na opisie przepływu użytkownika lub na diff'ie zmieniającym komponenty Angular. Dobry prompt zawiera opis przepływu w języku naturalnym: „Użytkownik wchodzi na `/login`, wypełnia formularz, klika `Zaloguj`, zostaje przekierowany na `/chat` i wysyła wiadomość."

Atrybuty `data-testid` powinny być dodane do komponentów Angular — agent może je zaproponować jako część zmian w PR.

Przykład wygenerowanego testu E2E Playwright

```
// tests/e2e/chat-flow.spec.ts
test.describe('Przepływ czatu AI', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill('test@example.com');
    await page.getByLabel('Hasło').fill('Test1234!');
    await page.getByRole('button', { name: 'Zaloguj' }).click();
    await expect(page).toHaveURL('/chat');
  });

  test('użytkownik wysyła wiadomość i otrzymuje odpowiedź AI', async ({ page }) => {
    await page.getByPlaceholder('Pytanie...').fill('Czym jest MongoDB?');
    await page.getByRole('button', { name: 'Wyślij' }).click();
    await expect(
      page.locator('[data-testid="message-assistant"]').last()
    ).toBeVisible({ timeout: 15000 });
  });

  test('przycisk Wyślij jest nieaktywny dla pustego pola', async ({ page }) => {
```

Pełna pętla CI — agent generujący i uruchamiający testy

```yaml
jobs:
  generate-tests:
    runs-on: ubuntu-latest
    if: github.event_name == 'pull_request'
    steps:
      - uses: actions/checkout@v4
        with:
          token: ${{ secrets.PAT_TOKEN }}
      - uses: actions/setup-node@v4
        with: { node-version: '22' }
      - run: npm ci
      - name: Generate tests via AI
        env:
          OPENAI_API_KEY: ${{ secrets.OPENAI_API_KEY }}
          PR_NUMBER: ${{ github.event.pull_request.number }}
        run: node agent/generate-tests.js
```

Job `run-tests` zależny od `generate-tests` uruchamia następnie `npm test` i `npx playwright test`.

Testy regresyjne — baseline i snapshot testing

Test regresyjny sprawdza, czy nowa zmiana nie zepsuła istniejącej funkcjonalności. Playwright oferuje wbudowane **snapshot testing** — porównywanie zrzutów ekranu z zapisanym wzorcem.

```
test('chat interface — snapshot regresyjny', async ({ page }) => {
  await page.goto('/chat');
  // Przy pierwszym uruchomieniu tworzy PNG baseline.
  // Przy kolejnych porównuje z baseline — fail przy różnicy >1%.
  await expect(page.locator('app-chat-container')).toHaveScreenshot(
    'chat-container-baseline.png',
    { maxDiffPixelRatio: 0.01 }
  );
});
```

```
# Aktualizacja baseline po celowej zmianie UI
npx playwright test --update-snapshots
```

Snapshot pliki przechowujemy w repozytorium — zmiany w wyglądzie UI są widoczne w PR jako diff obrazów.

Pokrycie kodu jako brama jakości merge'a

Jeśli pokrycie kodu spadnie poniżej progów, `npm test -- --coverage` kończy się kodem `1`, co automatycznie blokuje merge w GitHub Actions.

```
// jest.config.js
export default {
  coverageThreshold: {
    global: {
      branches:   70,
      functions:  80,
      lines:      80,
      statements: 80,
    },
  },
  collectCoverageFrom: [
    'backend/**/*.js',
    '!backend/**/*.test.js',
    '!backend/node_modules/**',
```

Progi można stopniowo podnosić w miarę wzrostu dojrzałości projektu. Brama pokrycia działa jak obiektywny, automatyczny strażnik jakości — niezależna od subiektywnej oceny recenzenta.

Mockowanie MongoDB w testach jednostkowych

Testy nie powinny łączyć się z prawdziwą bazą danych — `mongodb-memory-server` uruchamia MongoDB w pamięci procesu testowego:

```
import { MongoMemoryServer } from 'mongodb-memory-server';
import { MongoClient } from 'mongodb';

let mongod, client;

export async function startTestDB() {
  mongod = await MongoMemoryServer.create();
  client = new MongoClient(mongod.getUri());
  await client.connect();
  return client.db('testdb');
}

export async function stopTestDB() {
  await client.close();
```

Uruchamiamy `startTestDB()` w `beforeAll` i `stopTestDB()` w `afterAll`. Każdy test dostaje czystą bazę danych — brak zależności między testami, brak kosztów Atlas.

Mockowanie OpenAI API w testach jednostkowych

```
jest.mock('openai', () => ({
  default: jest.fn().mockImplementation(() => ({
    chat: {
      completions: {
        create: jest.fn().mockResolvedValue({
          choices: [{ message: { content: 'Mockowa odpowiedź AI' } }],
          usage: { prompt_tokens: 10, completion_tokens: 20, total_tokens: 30 },
        }),
      },
    },
  })),
}));
```

Mockowanie OpenAI sprawia, że testy są szybkie (milisekundy zamiast sekund), darmowe (brak wywołań API) i deterministyczne (zawsze ta sama odpowiedź). Pozwala też testować obsługę błędów przez `jest.fn().mockRejectedValue(new Error('API error'))`.

Podsumowanie — autonomiczny system testowania

Kompletny autonomiczny system testowania składa się z czterech warstw działających automatycznie przy każdym PR:

1. **Agent generujący testy** — pobiera diff, generuje pliki `.test.js` i scenariusze Playwright przez LLM, commituje je na gałąź PR.

2. **Uruchamianie testów** — Jest (jednostkowe + integracyjne z `mongodb-memory-server`) i Playwright (E2E z prawdziwą przeglądarką).

3. **Brama pokrycia** — blokuje merge jeśli pokrycie kodu spadnie poniżej zdefiniowanych progów.

4. **Testy regresji wizualnej** — snapshot testing Playwright wykrywa niezamierzone zmiany wyglądu UI.

Wyniki każdego uruchomienia archiwizowane są w MongoDB `test_runs` do analizy trendów jakości w czasie.